

Laboratório de Sistemas Operacionais

Prof. Angelo Elias Dal Zotto



Trabalho 2

Objetivos

- Expandir o conhecimento sobre construção e suporte de distribuições do sistema operacional Linux.
- Compreensão das estruturas de dados e interfaces internas do kernel Linux.
- Projeto e implementação de device drivers.

Descrição

Este trabalho consiste no projeto implementação de um módulo carregável de kernel (LKM – *Loadable Kernel Module*). Esse módulo terá como objetivo implementar um mecanismo para distribuição de mensagens entre processos utilizando o modelo Publish/Subscribe (acessível em `/dev/pubsub`). O sistema de mensagens poderá ser acessado por meio de uma múltiplos processos de uma aplicação que faz uso da API de acesso ao driver. Devem existir múltiplos processos em espaço de usuário, e cada um poderá realizar operações de leitura ou escrita no driver. Dessa forma, o driver será responsável por implementar a funcionalidade de um *broker*, ou seja, manter diferentes tópicos, manter os processos associados a cada tópico e distribuir mensagens.

As operações de escrita deverão enfileirar mensagens em uma lista gerenciada no próprio driver (em espaço de kernel), de tal forma que um processo possa consumir posteriormente mensagens enviadas ao driver e a ele endereçadas. Um processo é destino de uma mensagem se ele estiver inscrito em um tópico e uma mensagem for enviada a esse tópico, sendo que um processo poderá estar inscrito ou não em um ou mais tópicos.

Para o desenvolvimento do módulo carregável, deverão ser utilizadas as funções `kmalloc` e `kfree`, além da API de listas encadeadas disponíveis em nível de kernel. É importante que a gerência de memória seja implementada corretamente, evitando o travamento do sistema e vazamentos de memória.

Funcionamento

Após a sua carga, o módulo deverá atender solicitações de processos **implementados em uma aplicação** que usa funções da `stdio.h`, ou seja, não deve ser usado `cat` e `echo` para a entrega final. Antes de poder receber mensagens, um processo deve inscrever-se em um tópico. O módulo do kernel deve associar o processo realizando a escrita de `subscribe` ao

tópico, por meio de seu pid. Um processo poderá estar inscrito em diferentes tópicos. Para inscrever-se em um tópico teste, o processo deve usar uma operação de escrita em /dev/pubsub com o seguinte formato:

```
/subscribe teste
```

Isso cria um novo tópico em uma **lista de tópicos** caso o mesmo não exista. Caso o tópico já exista e o processo já estava inscrito anteriormente, o comando deve ser simplesmente ignorado.

Adicionalmente, um processo poderá remover sua inscrição em um tópico:

```
/unsubscribe teste
```

Esse comando remove todas as mensagens pendentes do tópico teste destinadas ao processo que realizou o unsubscribe. Além disso, também pode remover o tópico completamente caso não tenha mais processos inscritos nele.

Um processo pode enviar dados a outros processos também por meio de uma operação de escrita. Por exemplo, para enviar uma mensagem a um tópico com o nome teste, um outro processo pode realizar uma operação de escrita em /dev/pubsub com uma mensagem no seguinte formato:

```
/publish teste "Hello World!"
```

Se ainda não houver processos inscritos no tópico, a mensagem é simplesmente ignorada.

Diversas solicitações desse tipo podem ser enviadas por diferentes processos, sendo que cada tópico deve ter uma **lista de processos** inscritos. Cada processo dessa lista deverá ter uma **lista de mensagens** pendentes para leitura. Isso é necessário para que cada processo possa consumir a sua cópia da mensagem enviada ao tópico, cada um no seu tempo.

Para realizar a leitura de mensagens destinadas a um tópico, um processo deverá inicialmente realizar uma operação de escrita para configurar qual o tópico que será lido a seguir:

```
/fetch teste
```

Após a seleção do tópico um processo realizará operações de leitura em /dev/pubsub. Cada operação de leitura efetuada por um processo removerá uma única mensagem por vez da sua fila de mensagens do tópico. Caso o retorno seja 0, significa que não existem mais mensagens destinadas ao processo naquele tópico.

Ao fechar o arquivo (fclose() na aplicação), todos os recursos para o processo que disparou a fops de release devem ser liberados, ou seja, é necessário verificar todos os tópicos se o processo estava inscrito e realizar o processo de desinscrição, também removendo suas mensagens pendentes.

Parâmetros de configuração

O número máximo de tópicos deve ser configurado por um parâmetro através da macro module_param. Caso seja tentado criar um novo tópico, ultrapassando o número máximo de tópicos criados, a sua criação deve ser ignorada. Esse parâmetro não precisa ser modificável em tempo de execução.

Para cada tópico criado, deve ser mapeado um novo arquivo de configuração em /sys/pubsub/<nome_do_tópico>. Esse arquivo de configuração deve limitar o número máximo de processos inscritos no tópico criado, começando com um valor pré-definido

pelo usuário. Caso seja tentado realizar a inscrição de um novo processo ao tópico, ultrapassando o número máximo de inscritos, a sua inscrição deve ser ignorada. Caso seja alterado, em tempo de execução, o número máximo de processos inscritos ao tópico, para um número abaixo do número de processos atualmente inscritos, **não** devem ser excluídos processos inscritos.

Monitoramento de estatísticas

O driver deve criar um arquivo no `procfs /proc/pubsub` que imprime a contagem de mensagens publicadas desde o carregamento do módulo em cada tópico. Por exemplo, se 5 mensagens foram publicadas no tópico `teste` e 8 mensagens foram publicadas no tópico `sisop`, independentemente de elas já terem sido lidas ou não, a saída de `/proc/pubsub` deve ser:

`teste: 5`

`sisop: 8`

Tópicos que já foram removidos não devem apresentar estatísticas.

Condições de corrida

O acesso a estruturas compartilhadas pode causar condições de corrida. Para isso, **recomenda-se** que para iterar nas listas que podem ser simultaneamente modificadas por outros processos (seja para inserir ou para remover itens), seja garantido o acesso exclusivo por meio do uso de mutex. Uma única mutex protegendo o primeiro acesso a lista de tópicos, que é liberada somente ao fim da operação, pode ser suficiente, mas é mais eficiente separar em várias mutexes. A condição de corrida precisa ser evitada nos seguintes casos:

- Lista de tópicos: é necessário evitar que haja `/publish` em um tópico ao mesmo momento que outro processo realiza `/subscribe` ou `/unsubscribe`. Isso ocorre porque as operações de inscrição/desinscrição podem adicionar/remover tópicos da lista enquanto ela é iterada pelo `/publish`. Essa exclusão mútua também deve ocorrer na leitura do `procfs`, porque ele não pode acessar as estatísticas de um tópico que pode ser removido pelo `/unsubscribe`.
- Lista de processos de um tópico: é necessário evitar que o `/unsubscribe` e `/subscribe` modifiquem a lista de processos de um tópico ao mesmo tempo que ocorre a iteração do `/publish` para publicar a mensagem para todos os processos do tópico.
- Lista de mensagens de um processo: evitar que seja feita uma leitura ao mesmo tempo que é inserido uma mensagem por um `/publish`.

Dicas de implementação

- Obter o pid do processo que interage com o driver de dispositivo é feito usando a função `pid_t task_pid_nr(current)`, sendo `current` uma variável global fornecida pelo kernel que aponta para estrutura de dados do processo que disparou a operação de leitura ou escrita.
- É possível atribuir um ponteiro persistente ao longo das chamadas de `read/write` para cada processo, desde que o mesmo não chame a função `close/release`, com a intenção de salvar qual é o tópico que o processo está buscando mensagens, através do ponteiro `filep->private_data`.
- Para lançar múltiplos processos pelo terminal, é possível colocar a aplicação que está executando em *background* usando as teclas `Ctrl+Z` e então usar o comando `bg`. Dessa forma, é possível lançar mais uma aplicação enquanto a anterior está executando em *background*.

- Para listar os processos colocados em segundo plano, digite `jobs`. Para voltar a executar um processo em primeiro plano, utilize `fg %n` onde `n` representa o número do *job*.
- É possível organizar a implementação contendo uma lista global de tópicos, cada um contendo uma lista de processos inscritos. Cada processo inscrito pode ter uma lista de mensagens. Isso significa que irá haver uma cópia da estrutura de um processo para cada tópico inscrito e uma cópia da mensagem publicada para cada processo inscrito no tópico.
- O programa de teste pode ser implementado como um terminal interativo para o `/dev/pubsub` que lê uma linha e envia o comando da linha para o dispositivo, sem que o arquivo seja fechado e re-aberto entre os comandos. Uma leitura pode ser implementada por meio de um comando `/read` no programa de teste, que realiza então uma **leitura** do `/dev/pubsub`.
- Recomenda-se uma mutex global para lista de tópicos, uma mutex para cada tópico controlar acesso a sua lista de processos, e uma mutex para cada processo controlar acesso a suas mensagens.
- Cuidado para não ocorrer deadlock. Se realmente precisar obter mais de um mutex ao mesmo tempo (ex: o do tópico, o do processo e o da mensagem), garanta que todos os os processos envolvidos peçam a mutex na mesma ordem.

Uma mutex no kernel Linux é definida por uma `struct mutex` incluída por `linux/mutex.h`. Ela deve ser inicializada usando a função `mutex_init(&mtx)` e não precisa ser deinicializada caso não seja mais usada. Para adquirir a mutex (bloquear), usa-se `mutex_lock(&mtx)`, e para desbloquear `mutex_unlock(&mtx)`.

A documentação de mutex em espaço de kernel e um exemplo de seu uso estão disponíveis no Moodle.

Entrega

Este trabalho deverá ser realizado em grupos de 3 ou 4 integrantes (iguais ao do T1, indicados no Moodle) e apresentado individualmente no dia 13/05 (apresentação em torno de 5 minutos). Para a entrega, é esperado que apenas um dos integrantes envie pelo Moodle um arquivo compactado do projeto com o nome dos integrantes, contendo os scripts criados (arquivo `.config` e pasta `custom-scripts`, por exemplo) e os arquivos contendo o código fonte da aplicação implementada (pasta `apps`, por exemplo) e o código fonte do módulo implementado (pasta `modules`, por exemplo). Uma estrutura do `.zip` criado para entrega pode se parecer com a abaixo:

```
.
├── apps
│   └── pubsub-teste.c
├── custom-scripts
│   ├── network-config
│   ├── pre-build.sh
│   └── qemu-ifup
├── modules
│   └── pubsub
│       ├── Makefile
│       └── pubsub.c
└── .config
```

Avaliação

Peso	Descrição
20%	Funcionamento da inscrição de processos aos tópicos, criando tópicos caso não existam
20%	Funcionamento da publicação de mensagens em processos inscritos em tópicos
10%	Remoção de mensagens publicadas em processos, sem vazamentos de memória
10%	Remoção de processos inscritos em tópicos, que também remove todas suas mensagens pendentes, e de tópicos caso não tenham mais processos inscritos, sem vazamentos de memória
10%	Liberação de recursos do processo (desinscrição do processo em todos os tópicos inscritos) na função release
10%	Configuração de limitação do número máximo de processos inscritos em cada tópico individual durante tempo de execução usando o sysfs
10%	Implementação correta de sincronização sem deadlocks
5%	Configuração por parâmetro de número máximo de tópicos durante o carregamento do módulo
5%	Impressão correta de estatísticas no procfs